# Department of Computer Science and Information Technology

| | | | | | |
|---|---|---|---|---|---|
| **Name** | | | | **Id** | |
| **Course** | CSCI426 | | | | |
| **Date** | 10 Dec 2024 | **Time** | 08:00 – 08:45 | **Duration** | 45 minutes |
| **Pages** | 2 | **Calculators** | | **Documents** | |

The following database structure is provided to you in the file sql.txt. Ensure your implementation works with this schema.

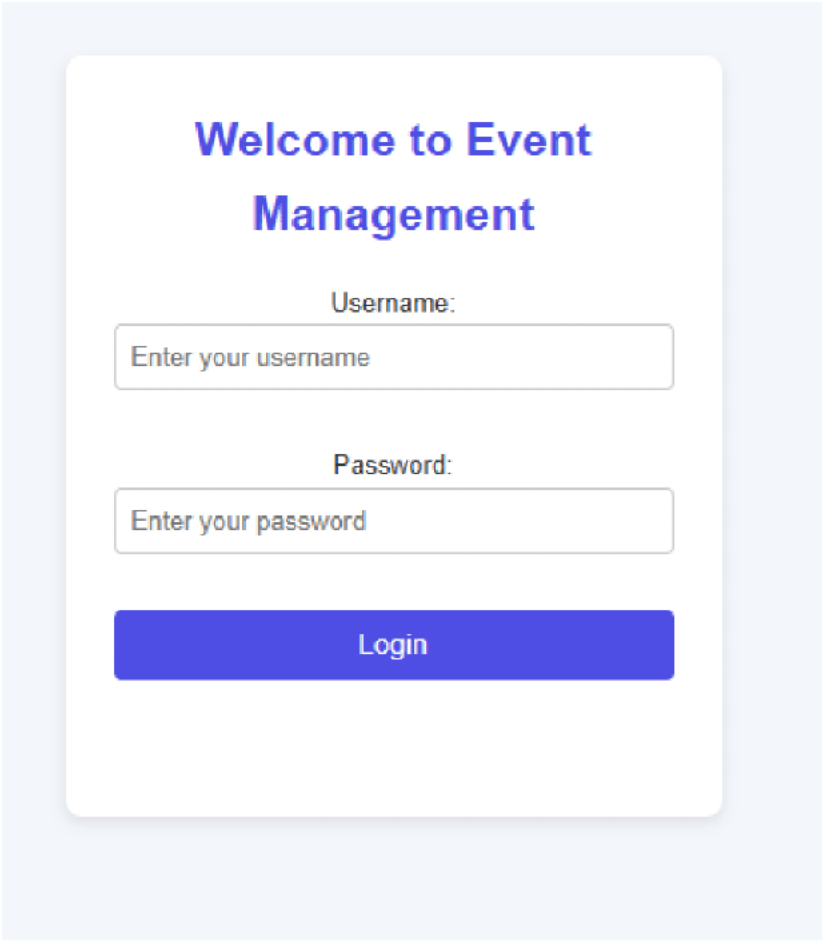
**Database Name:** event_management

## Global Instructions

1. **Database Connection**:
   - Establish a database connection using mysqli in all pages.
   - Ensure to check for connection errors and terminate the script with an error message if the connection fails.
2. **Session Management**:
   - Use PHP sessions to manage user authentication and role-based access control.
   - Restrict access to pages (admin_dashboard.php, user_dashboard.php, add_event.php) based on the user's role.
3. **Error Handling**:
   - Handle database errors gracefully and display appropriate error messages to the user.
4. **Security**:
   - Use real_escape_string to sanitize user inputs before executing database queries.

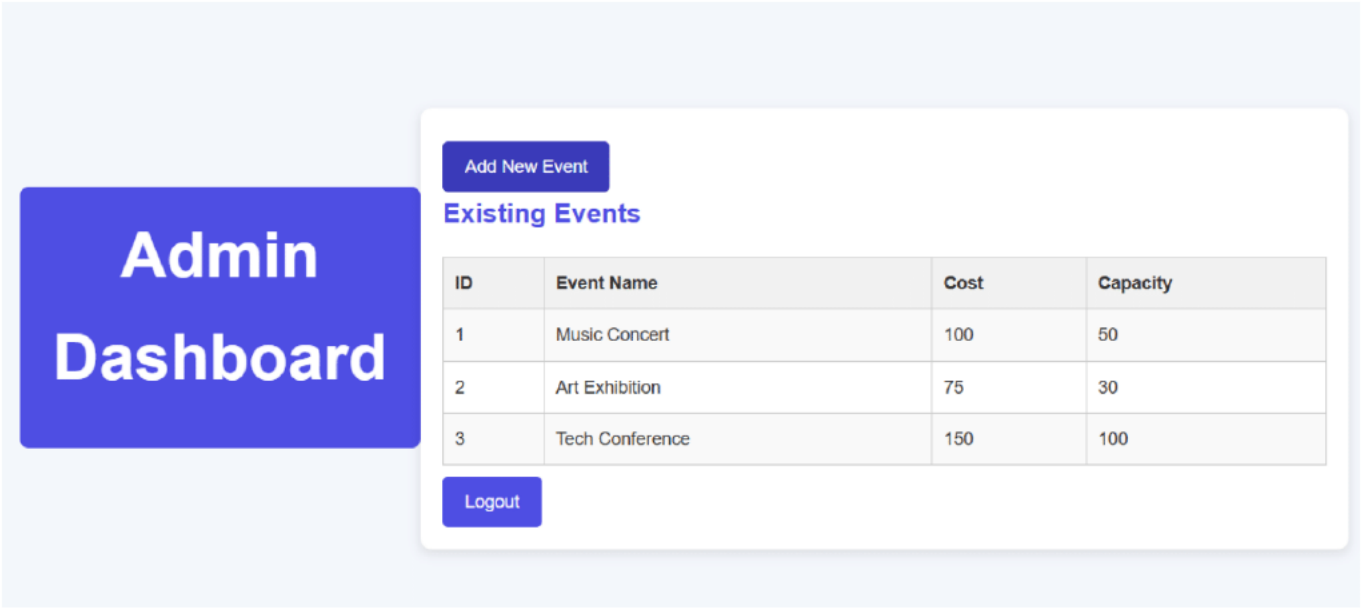
### 1. Login Page (index.php)

- **Purpose**:
  - Authenticate users based on their username and password.
  - Redirect users to their respective dashboards (admin_dashboard.php for Admin and user_dashboard.php for User).
- **Requirements**:
  - Create a login form with fields for username and password.
  - Check the credentials against the users table.
  - Store the username, role, and user_id in the session upon successful login.
  - Display an error message if the credentials are invalid.



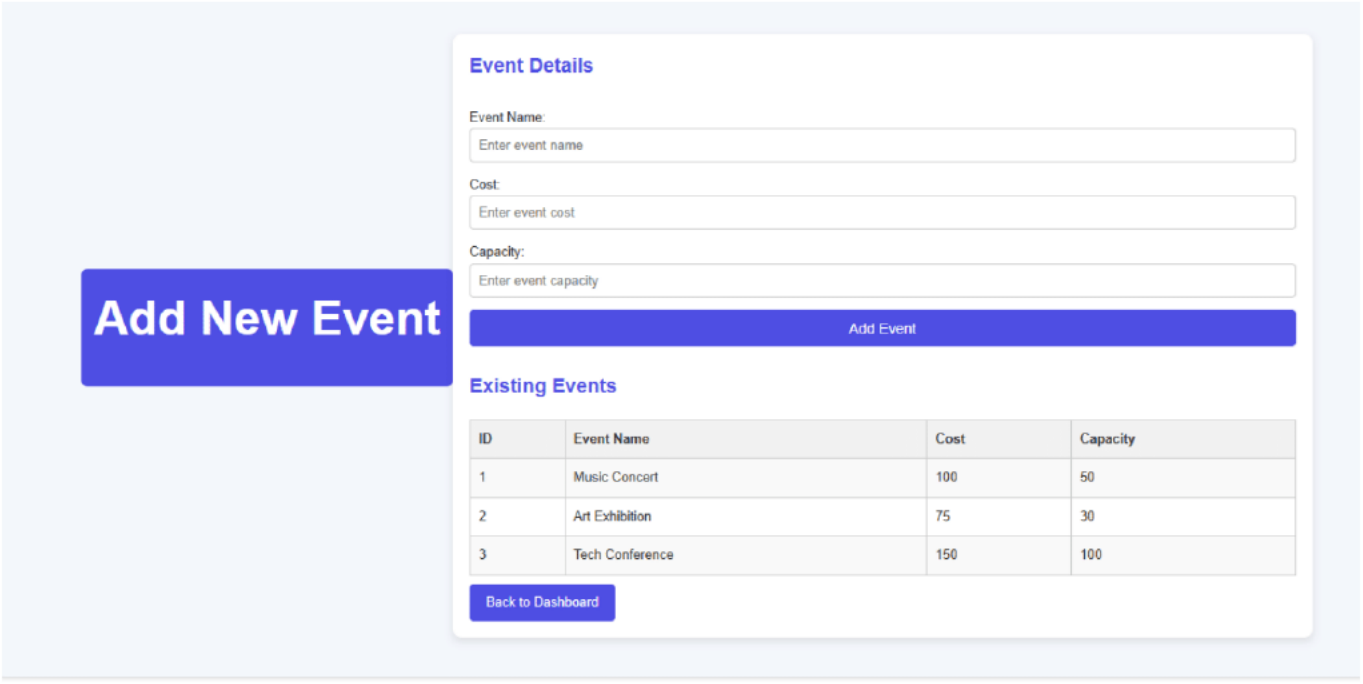

**2. Admin Dashboard (admin_dashboard.php)**

- **Purpose**:
  - Display all events to the admin.
  - Provide a link to the add_event.php page to add new events.
- **Requirements**:
  - Fetch and display all events from the events table in a table format with columns for:
    - Event ID
    - Event Name
    - Cost
    - Capacity
  - Add a button or link to navigate to add_event.php.
  - Include a logout button to end the session and redirect to the login page.



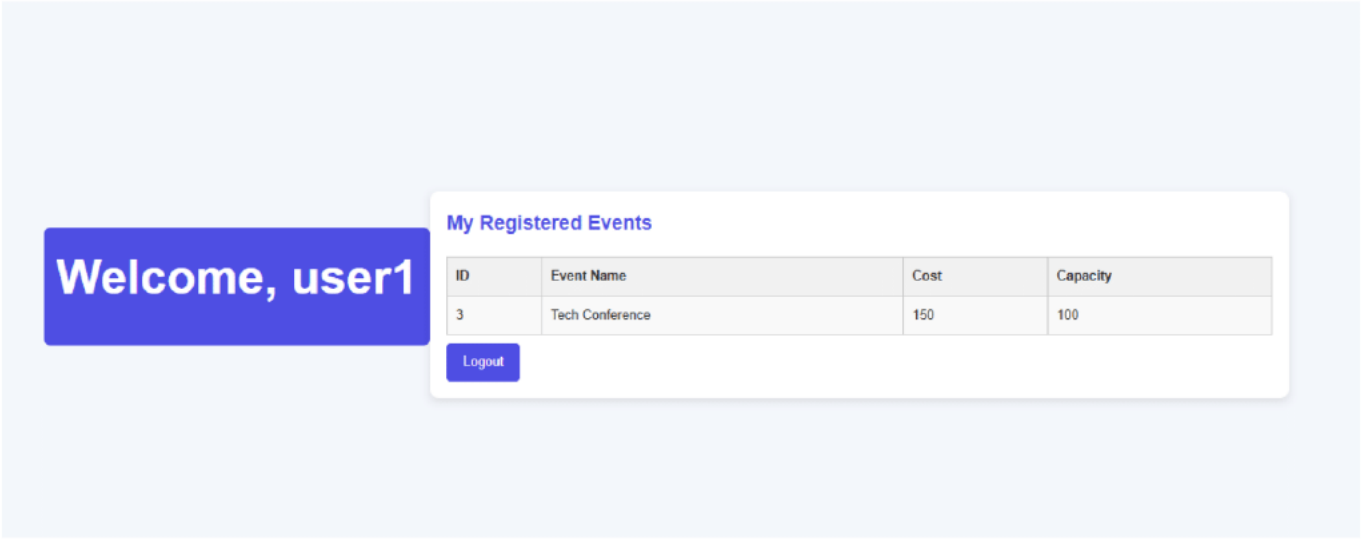

### 3. Add Event Page (add_event.php)

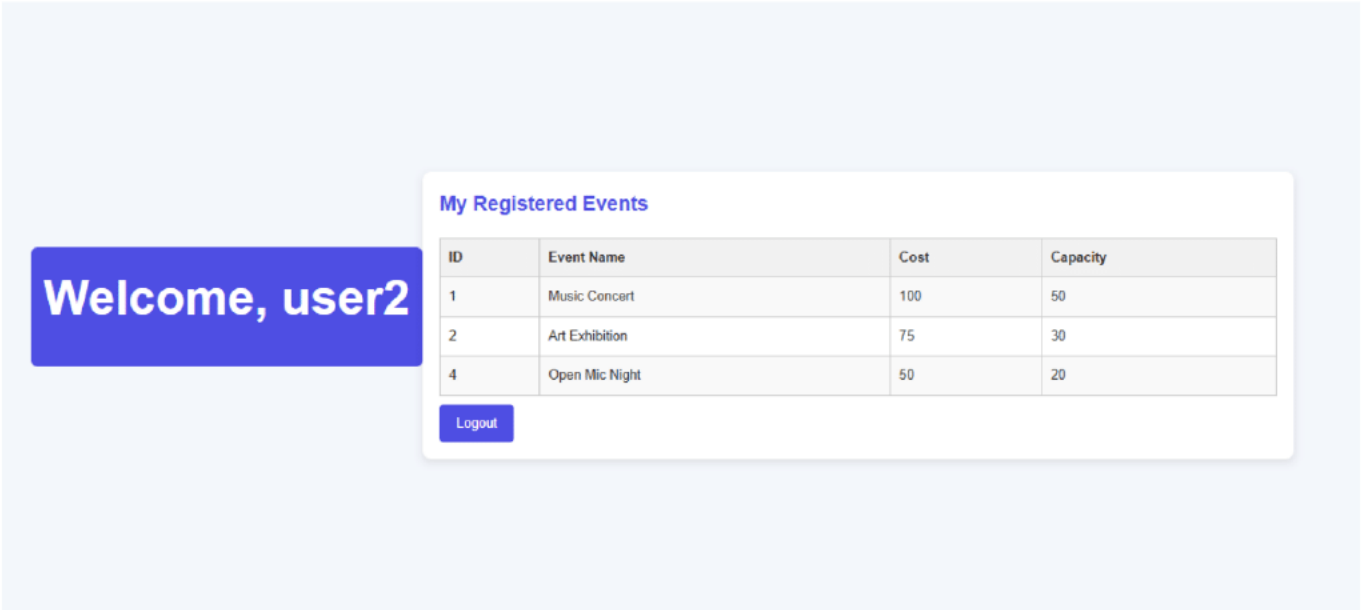
- **Purpose**:
  - Allow the admin to add new events to the system.
- **Requirements**:
  - Create a form with fields for:
    - Event Name
    - Cost
    - Capacity
  - Upon submission, validate the inputs and insert the new event into the events table.
  - Display a success message upon successful insertion.
  - Show an error message if the insertion fails.
  - Below the form, display a list of all existing events in a table format.



**4. User Dashboard (user_dashboard.php)**

- **Purpose**:
  - Display events registered by the logged-in user.
- **Requirements**:
  - Fetch and display events from the events table where the user_id matches the logged-in user's ID.
  - Display the events in a table format with columns for:
    - Event ID
    - Event Name
    - Cost
    - Capacity
  - Include a logout button to end the session and redirect to the login page.



**5. Logout Page (logout.php)**

- **Purpose**:
  - End the session and redirect the user to the login page.
- **Requirements**:
  - Destroy the session.
  - Redirect the user to index.php after logging out.
  - Display a confirmation message that the user has been logged out.

**Notes**

- Make sure to test your application thoroughly.
- Use meaningful variable names and write clean, readable code.
- Submit your completed project as a ZIP file containing all the files (index.php, admin_dashboard.php, add_event.php, user_dashboard.php, logout.php, and styles.css).

Good luck ☺